



MC-LRNN: A logic-based neural network for multi-class software vulnerability prediction[☆]

Yuxiang Shang^{a,b}, Shaoying Liu^a,^{*}

^a Software Engineering Institute, East China Normal University, Shanghai, 200062, Shanghai, China

^b Graduate School of Advanced Science and Engineering, Hiroshima University, Higashihiroshima, 739-8511, Hiroshima, Japan

ARTICLE INFO

Keywords:

Logic Rule

Neural network

Vulnerability prediction

ABSTRACT

Software vulnerabilities are a major threat to information systems. Detecting them early and accurately is critical. Software metrics are commonly used in vulnerability prediction, but choosing the most relevant features remains a major challenge. In this paper, we present Multi-Class Logic Rules Neural Network (MC-LRNN), a novel model that combines logic-based reasoning with neural networks for software vulnerability prediction. MC-LRNN uses a Top-Down Hill-Climbing Greedy Algorithm to extract first-order logic rules from software metrics, forming an interpretable reasoning layer that guides the learning process. The dataset is divided into a Logic Rule Dataset for rule generation and a Learning Dataset for model training and evaluation.

We evaluate MC-LRNN on three benchmark datasets — Juliet, SARD, and REVEAL — under both binary and multi-class classification settings. The results show that MC-LRNN consistently outperforms traditional baselines, handles class imbalance effectively, and generalizes well across projects. Its design provides both interpretability and strong generalization capabilities, making it well-suited for real-world vulnerability prediction. Code and datasets are available at: <https://github.com/Seansyx123/LRNN>.

1. Introduction

Software vulnerabilities are unavoidable due to the inherent difficulty of writing secure code, which demands substantial expertise. Even skilled developers are prone to human errors, potentially resulting in programming mistakes with serious implications for information security. Early prediction of such vulnerabilities can substantially mitigate these risks. However, manually identifying vulnerable code is a complex, time-intensive, and expensive task. To address these challenges and reduce costs, automated tools that predict vulnerable source entities (e.g., files, functions) have become indispensable. The field of automatic vulnerability prediction (AVP) (Zagane et al., 2020) has garnered significant research attention.

Early Vulnerability Prediction Models (VPMs) rely on software metrics to classify source code entities into categories such as vulnerable and clean (binary classification), or good, bad, and mixed (multi-class classification). These metrics, which capture properties like complexity, coupling, and size, have shown strong correlations with the presence of software vulnerabilities (Kalouptoglou et al., 2023). Specialized software metrics for vulnerability prediction have also been proposed, as detailed in Shin et al. (2010), with partial examples presented in Table 1. The advantage of this traditional approach lies in its inherent

interpretability; however, due to its relatively low performance and the effort required to generate appropriate software metrics, it has gradually fallen out of favor.

Recent studies (Chakraborty et al., 2021) have investigated AVP using deep learning (DL) techniques. DL-based AVP models have predominantly relied on text mining. For example, software vulnerability prediction models based on RNN (Nguyen et al., 2021), LSTM (Xu et al., 2018), and CNN (Guo et al., 2023) architectures. With the advancement of computational power, an increasing number of software vulnerability prediction methods based on large language models (LLMs) have been proposed, such as CodeBERT (Feng et al., 2020a), CodeGPT and others (Kalouptoglou et al., 2024). These models have demonstrated high performance in software vulnerability prediction tasks. However, they suffer from two main limitations: First, deep learning is inherently a black-box approach and lacks interpretability, making it difficult to provide actionable insights or guidance for developers during subsequent programming. Second, the results of deep learning models are often sensitive to class imbalance. In real-world scenarios, software vulnerability prediction typically involves highly imbalanced data, where clean samples significantly outnumber vulnerable ones. For

[☆] Editor: Yan Cai.

^{*} Corresponding author.

E-mail addresses: shangyuxiang@hiroshima-u.ac.jp (Y. Shang), syliu@sei.ecnu.edu.cn (S. Liu).

Table 1
Samples of software metrics for vulnerability prediction.

Metric	Definition
CountLineCode	The number of lines of code in a file
CountDeclFunction	The number of functions defined in a file
CountLineCodeDecl	The number of lines of code in a file devoted to declarations
CountLinePreprocessor	The number of lines of code in a file devoted to preprocessing
SumEssential	The sum of essential complexity, where essential complexity is defined as the number of branches after reducing all the programming primitives such as a for loop in a function's control flow graph into a node iteratively until the graph cannot be reduced any further
SumCyclomaticStrict	The sum of the strict cyclomatic complexity, where strict cyclomatic complexity is defined as the number of conditional statements in a function
SumMaxNesting	The sum of the MaxNesting in a file, where MaxNesting is defined as the maximum nesting level of control constructs such as if or while statements in a function
MaxCyclomaticStrict	The maximum of strict cyclomatic complexity in a file
MaxMaxNesting	The maximum of MaxNesting in a file
SumFanIn	The sum of FanIn, where FanIn is defined as the number of inputs to a function such as parameters and global variables
SumFanOut	The sum of FanOut, where FanOut is defined as the number of assignment to the parameters to call a function or global variables
MaxFanIn	The maximum of FanIn
MaxFanOut	The maximum of FanOut
CommentDensity	The ratio of lines of comments to lines of code

example, in the REVEAL dataset (Chakraborty et al., 2021), vulnerable samples account for less than 10% of the total.

This study presents a novel neural network model, the Multi-Class Logic Rules Neural Network (MC-LRNN), which incorporates logic rules based on software metrics and leverages these information for feature selection. This approach preserves the interpretability of software metric-based prediction methods while leveraging the high performance of neural network models. At the same time, the feature selection based on logic rules identifies attributes that are highly correlated with vulnerable samples, which helps mitigate the impact of class imbalance. Prior research (Shang and Liu, 2024) demonstrates that Logic Rules Neural Network (LRNN) outperforms existing methods in software defect prediction tasks by addressing class imbalance and facilitating effective cross-project predictions. This work extends the original approach to address multi-class classification problems in software vulnerability prediction, and is therefore referred to as the Multi-Class Logic Rules Neural Network (MC-LRNN).

Software vulnerability prediction is more challenging than general defect prediction due to its focus on security, the need for complex features, and limited data availability. The results in this paper show that MC-LRNN effectively tackles this problem. It supports both binary and multi-class classification, reduces the impact of class imbalance often seen in real-world data, and maintains strong interpretability. We conducted extensive experiments on the REVEAL (Chakraborty et al., 2021), SATE IV Juliet (Okun et al., 2013) and SARD (Black et al., 2017) datasets, employing multiple performance metrics to evaluate the effectiveness of our proposed model, as detailed in Section 4. The key contributions of this paper are as follows:

- Introduction of the Multi-Class Logic Rules Neural Network (MC-LRNN), a novel neural network model that integrates logic rules and is specifically optimized for vulnerability prediction tasks.
- The MC-LRNN model is evaluated on multiple datasets, demonstrating its effectiveness in handling class imbalance and performing feature selection.
- Expanding the application scenarios of the Multi-Class Logic Rules Neural Network (MC-LRNN) from binary classification to multi-class classification.

The remainder of this article is organized as follows: Section 2 offers an overview of related techniques in vulnerability prediction. Section 3 describes the algorithmic framework and methodology of the proposed MC-LRNN model. Section 4 presents an analysis of the experimental results and Section 5 is the discussion of our approach, while Section 6 concludes the paper by discussing potential directions for future research.

2. Related work

This section provides an overview of the background and emphasizes closely related works in the field. It begins by outlining the context of the study, introducing key concepts such as vulnerability prediction based on text mining, vulnerability prediction based on software metrics, the integration of logic rules with neural networks and mitigating class imbalance effects through feature selection. The section concludes by referencing the most relevant related works.

2.1. Vulnerability prediction based on text mining

Deep neural networks tailored for text mining, commonly applied in natural language processing (NLP), include models such as recurrent neural networks (RNNs) (Nguyen et al., 2021), convolutional neural networks (CNNs) (Guo et al., 2023), and long short-term memory networks (LSTMs) (Xu et al., 2018). In the context of vulnerability detection, these models can be leveraged by treating source code as natural language text, enabling the application of text-based neural architectures for code analysis. Poster (Lin et al., 2017) is a vulnerability detection framework that utilizes abstract syntax trees (ASTs) to extract sequential features from programs, enabling function-level vulnerability detection across software projects. VulDeePecker (Li et al., 2018) is a slice-level vulnerability detection model based on bidirectional long short-term memory networks (BLSTM). Russell et al. (2018) designed a deep learning-based vulnerability detection tool that relies on feature representations. VulSniper (Duan et al., 2019) is a vulnerability detection model that employs feature tensor encoding. Feng et al. (2020b) proposed a vulnerability detection tool based on abstract syntax trees. μ VulDeePecker (Zou et al., 2019) is a significant enhancement of VulDeePecker, extending it to support multi-type vulnerability detection. Lin and Cai (2021) conducted a series of empirical studies comparing the effectiveness of BLSTM and the emerging BGRU models in real-world vulnerability detection tasks. Qiang (2022) proposed a software structure detection algorithm based on an improved convolutional neural network (CNN) model. Building upon VulDeePecker, Li et al. (2021) further refined its vulnerability slicing method and introduced SySeVR. VulCNN (Wu et al., 2022) is a vulnerability detection model based on convolutional neural networks for image classification. VulBG (Yuan et al., 2023) is a neural network-based vulnerability detection framework that incorporates behavior graph modeling.

Recently, large language models (LLMs) have been applied to software vulnerability prediction. Trained on large-scale code corpora,

they capture deep semantic and syntactic patterns without manual feature engineering (Feng et al., 2020a; Chen et al., 2021). Common pre-trained language models include Word2Vec (Church, 2017), GloVe (Pennington et al., 2014), and CodeBERT (Feng et al., 2020a). Among them, CodeBERT, as a pre-trained model specifically designed for programming languages, is relatively recent and widely used as a code embedding model. These models outperform traditional DL approaches in many tasks due to their strong representation capabilities.

2.2. Vulnerability prediction based on software metrics

Metric-based vulnerability prediction models utilize manually designed software features — such as code complexity, code churn, developer activity, coupling, and cohesion — to characterize code components. These features are then fed into machine learning classifiers, including logistic regression (LR), support vector machines (SVM), decision trees (DT), random forests (RF), naïve Bayes (NB), and Bayesian networks (BN) to identify and predict vulnerabilities.

Shin and Williams (2008) conducted a study on Mozilla's JavaScript engine and demonstrated that nine complexity metrics, including McCabe and SLOC, can distinguish vulnerable from non-vulnerable functions, though their method suffered from a relatively high false positive rate. Gegick et al. (2008) employed source code and change metrics in a regression tree model, achieving 100% recall and 8% false positives under optimal conditions. Meneely and Williams (2009) explored the effect of developer collaboration, finding that files modified by nine or more developers are 16 times more likely to be vulnerable. Chowdhury and Zulkernine (2011) evaluated 28 security metrics and found that 24 could effectively identify files in one or two attack-prone categories, offering practical guidance for inspection and testing. Chowdhury and Zulkernine (2011) proposed a framework combining complexity, cohesion, and coupling metrics, which successfully detected most vulnerable files in Firefox with a low false positive rate.

2.3. Integration of logic rules with neural networks

As neural networks gain increasing popularity, their limitations have become more apparent, with reduced robustness and interpretability ranking among the most critical challenges. To address these issues, integrating domain knowledge into neural network models has emerged as a key research focus (Dash et al., 2022). Neuro-symbolic AI (Hitzler and Sarker, 2022) has been proposed as a promising approach, bridging the gap between neural networks and logic rules.

First-order logic, a fundamental method for knowledge representation, has been explored for integration into machine learning. This exploration led to the development of Inductive Logic Programming (ILP) (Muggleton and De Raedt, 1994). Unlike black-box models, ILP (Cropper and Dumančić, 2022) utilizes domain knowledge expressed in first-order logic, providing a transparent and interpretable approach to problem-solving.

Recently, neural networks that integrate logic rules have garnered increasing attention. Hu et al. (2016) proposed an iterative distillation method that embeds structured information from logic rules into a neural network's weights. Similarly, Li and Srikumar (2019) and Qu and Tang (2019) investigated the incorporation of logic rules into various neural network architectures, including convolutional neural networks, recurrent neural networks, and deep neural networks, focusing on their input and output layers.

IBM Research (Fagin et al., 2020) introduced a reasoning-capable neural network called Logical Neural Networks (LNN). The MC-LRNN model presented in this paper builds upon the LNN framework. LNN incorporates logic rules as constraints that regulate the neural network's activation functions, effectively guiding the model's outputs in alignment with logical reasoning principles.

2.4. Mitigating class imbalance effects through feature selection

Feature selection and class imbalance are widely recognized challenges in software vulnerability prediction. However, no existing research in this field has specifically used feature selection to address class imbalance. While related studies exist in other domains, scholars across various disciplines have investigated the impact of feature selection on mitigating class imbalance. First, Chen and Wasikowski (2008), Wasikowski and Chen (2009) addressed class imbalance in small samples through feature selection. Then, Yin et al. (2013), Maldonado et al. (2014), Maldonado and López (2018), and Zhou et al. (2017) explored feature selection techniques to address the class imbalance problem in high-dimensional data. Liu et al. (2019) and Kamalov et al. (2023) proposed a new feature selection method to address the classification challenges posed by imbalanced data. Matharaarachchi et al. (2021) assessed the relationship between class imbalance and feature selection.

Compared to previous studies, this paper integrates logic rules into neural networks using symbolic AI techniques. These rules guide feature selection, thereby mitigating the effect of class imbalance and enhancing software vulnerability prediction.

3. Methodology

In this paper, we propose a reasoning neural network model, called MC-LRNN, for software vulnerability prediction. It is a classification neural network based on software metrics, where the input consists of metric information extracted from the target project, and the output indicates whether the corresponding component is vulnerable. The architecture of the proposed MC-LRNN model is illustrated in Fig. 1.

Initially, the source software project is randomly partitioned into two subsets: the Logic Rule Dataset and the Learning Dataset. The Logic Rule Dataset is employed to derive logic rules, while the Learning Dataset is used for training and evaluating the neural network model. The data from the Logic Rule Dataset is first fed into the LRNN module to generate logic rules, which are then used for reasoning—that is, identifying meaningful software metrics to guide feature selection. The Learning Dataset serves as the traditional component for classifier training, consisting of both a training set and a testing set. However, the attributes in its input layer are determined based on the feature selection results derived from the logic rules.

The Logical Neural Network (LNN) (Riegel et al., 2020) seamlessly integrates neural network-based learning with logic rule reasoning techniques. LNNs utilize rule-based parameters to adjust the thresholds of activation functions, allowing logical rules to influence the neural network's outputs. In the proposed method, the LNN is incorporated as a module to facilitate reasoning about logical statements. The architecture of LRNN module includes a rule generation layer preceding the LNN layer, as detailed in Section 3.1. This layer applies a rule-learning technique to analyze the software metrics provided as input. Following the LNN layer, an attribute selection layer filters the reasoning results, identifying software metrics associated with vulnerable samples. These metrics are subsequently used as input attributes for the neural network classifier. This pipeline ensures the seamless integration of logical rules with the neural network, enabling the model to effectively combine reasoning and learning-based approaches for vulnerability prediction.

In summary, numerous software metrics are employed to evaluate code within three datasets: REVEAL (Chakraborty et al., 2021), SATE IV Juliet (Okun et al., 2013) and SARD (Black et al., 2017). Utilizing these software metrics directly as attributes in a deep learning prediction model results in low predictive capability. The MC-LRNN model initially processes a segment of the code from the source project, employing rule learning to formulate a logical proposition. The antecedents of these logical propositions are the software metrics and their corresponding values. The consequent of these logical propositions indicates whether the software is vulnerable. The LNN module within the MC-LRNN deduces the generated logic rules and selects the

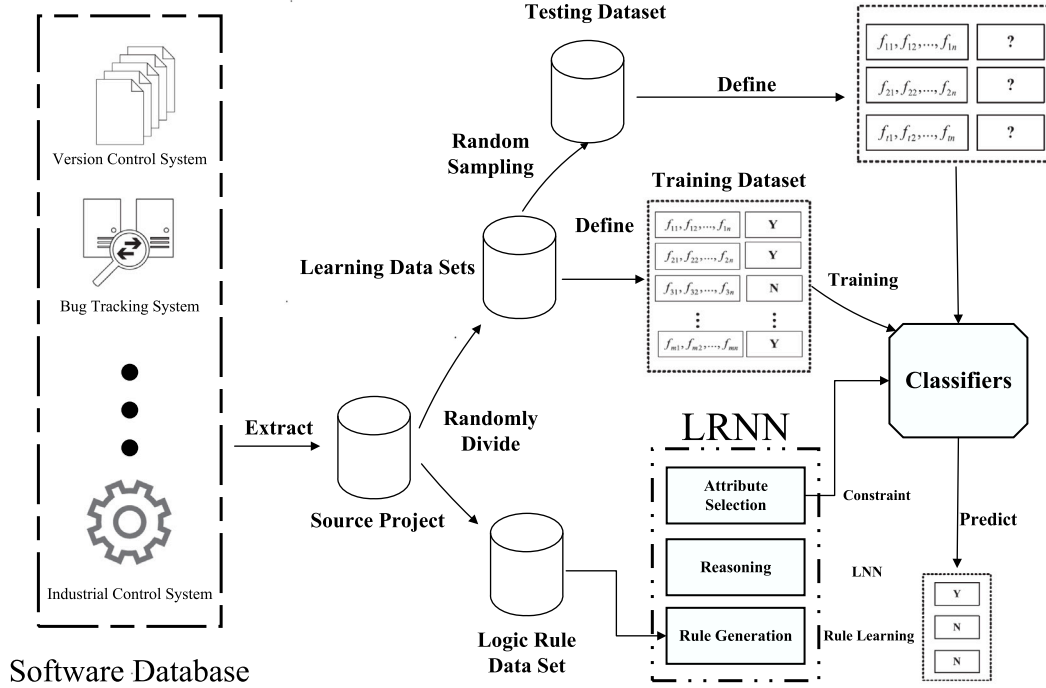


Fig. 1. Structure of MC-LRNN Approach.

software metrics most closely associated with vulnerable software. Ultimately, only these selected software metrics are employed as attributes in the neural network classifiers. This integrates the information as constraints into the neural network. Specific technical details are given below.

3.1. Rule learning

In machine learning, a “rule” refers to a semantically interpretable pattern that describes the distribution of data. These rules are typically expressed as logical statements in the “IF...THEN...” format (Fürnkranz et al., 2012). Rule learning involves deriving such rules from a training dataset to distinguish between different examples (Zhou, 2021). Rule learning can be categorized into two primary areas (Fürnkranz and Kliegr, 2015): Descriptive Rule Learning: Focuses on uncovering regularities within specific parts of the dataset, providing insights into the data’s structure. Predictive Rule Learning: Aims to generalize from the dataset to make accurate predictions on unseen data. This paper emphasizes descriptive rule learning, with the goal of identifying meaningful patterns and regularities in the dataset to support reasoning and enhance the model’s interpretability.

3.1.1. Generation of logic rules

Algorithm 1 function FINDPREDICTIVERULE (Examples)

```

1: Input: Examples, a set of positive and negative examples for a class  $c$ .
2: //initialize the rule body
3:  $rb \leftarrow \emptyset$ 
4: // repeatedly find the best refinement
5: repeat
6:   build refinements  $R \leftarrow \{rb' \mid rb' = rb \wedge f, \text{ for some feature } f\}$ 
7:   evaluate all  $rb' \in R$  according to some quality criterion
8:    $rb = \text{the best refinement in } R$ 
9: until  $rb$  satisfies a stopping criterion or covers no examples
10: Output: rule ( $c \leftarrow R$ )

```

In descriptive rule discovery, the primary objective is to identify regularities within a given dataset. One of the most widely used algorithms for this purpose is the Top-Down Hill-Climbing Algorithm (Fürnkranz, 2002). Algorithm 1 depicts a greedy hill-climbing algorithm designed to discover a single predictive rule. The algorithm starts with an empty rule body and iteratively adds new conditions. Each condition is evaluated based on a heuristic quality criterion, which typically considers: The number of examples covered and uncovered by the rule. Whether the covered examples belong to the target class “c” (positive examples) or do not belong to class “c” (negative examples). The algorithm continues adding conditions until no positive examples remain uncovered. This process produces a complete set of rules for the dataset, capturing meaningful regularities that distinguish the target class from non-target classes.

To illustrate the rule generation process more clearly, we present a step-by-step example using the REVEAL dataset, which includes simple function-level software metrics and binary vulnerability labels. Each instance is described by attributes such as Lines of Code (LOC), Cyclomatic Complexity (CC), and Number of Function Calls (NFC).

In the initial iteration, the algorithm performs a greedy search over the feature space to find a simple rule that best separates the vulnerable and non-vulnerable samples. Suppose the metric LOC is found to be weakly predictive. A preliminary rule might be induced:

Rule 1 : IF LOC > 1 THEN vulnerable (1)

This rule has low complexity and is generated due to its coverage of a large portion of vulnerable samples in the early phase. However, in subsequent iterations, the algorithm evaluates alternative thresholds to improve precision and reduce false positives. For instance:

Rule 2 : IF LOC > 250 THEN vulnerable (2)

Here, the threshold has been adjusted to increase the rule’s discriminative power. If Rule 2 achieves a higher confidence (e.g., higher ratio of true positives among covered samples), it replaces Rule 1.

To further refine the rule, the algorithm may explore conjunctions with additional attributes. For example:

Rule 3 : IF LOC > 250 AND CC > 10 THEN vulnerable (3)

This compound rule captures a more specific pattern by combining multiple metrics, thereby improving both precision and interpretability. The process continues iteratively, adding or removing conditions (e.g., using hill-climbing) until a stopping criterion is met—such as reaching a maximum rule set size, or when no further improvement in performance is achieved.

In the final rule set, redundant or low-confidence rules are pruned, and only the most informative rules are retained. Thus, the initial simple rule evolves through threshold refinement, feature conjunction, and iterative replacement, resulting in a set of interpretable logic rules that generalize well across the dataset.

Foil gain measures the extent of data extraction into rules and is expressed as:

$$\text{FoilGain} = \hat{n}_+ \times (\log_2 \frac{\hat{n}_+}{\hat{n}_+ + \hat{n}_-} - \log_2 \frac{n_+}{n_+ + n_-}) \quad (4)$$

Here, $\hat{n}_+$, $\hat{n}_-$ are the counts of positive and negative examples covered by the new rules after incorporating candidate data. Similarly, n_+ , n_- represent the counts of positive and negative examples covered by the original rules.

Greedy hill-climbing algorithms often produce a number of rules proportional to the dataset size, with each rule likely corresponding to a single data point. Without appropriate constraints, this process may result in the continuous generation of rules, increasing the risk of overfitting. To mitigate this, rule refinement techniques are crucial for controlling rule creation and preventing excessive specificity.

3.1.2. Refinement of logic rules

Pruning is essential to mitigate overfitting caused by greedy algorithms. The quality of generated rules is assessed using Likelihood Ratio Statistics (LRS), defined as:

$$\text{LRS} = 2 \times (\hat{m}_+ \log_2 \frac{\hat{m}_+}{\hat{m}_+ + \hat{m}_-} + \hat{m}_- \log_2 \frac{\hat{m}_-}{\hat{m}_+ + \hat{m}_-}), \quad (5)$$

where m_+ , m_- represent the number of positive and negative examples in the training set, and $\hat{m}_+$, $\hat{m}_-$ are the corresponding counts in the logic rules set. The LRS measures the difference between the data distribution in the training set and the rule set.

A larger LRS value suggests that additional rules can be derived from the dataset, while a smaller value indicates limited regularity. To further refine the generated rules, Reduced Error Pruning (Fürnkranz and Widmer, 1994) is employed. The LRS also enables regulation of the number of generated rules, balancing complexity and generalization.

In traditional rule learning, the refinement metric is treated as a hyperparameter, requiring manual configuration. The predictive accuracy heavily depends on this parameter's setting, which represents a key limitation in using rule learning for prediction tasks. Our approach addresses this limitation by separating responsibilities: rule learning serves as a module for generating logical rules, while a neural network is used for making predictions. This design reduces the dependency on manually tuned parameters, enhances robustness, and mitigates the drawbacks of rule learning.

In the Experiments section, we directly utilize the Python implementation of RuleKit¹ (Gudyś et al., 2024), a versatile and open-source rule-learning toolkit developed by Gudyś et al. (2020). RuleKit supports various rule learning tasks, including classification, regression, and survival analysis. In our study, RuleKit is used to generate interpretable logic rules from metric-based software vulnerability datasets. We configure RuleKit to use greedy hill-climbing search and Foil Gain as the rule evaluation criterion. Its parameters (e.g., minimum rule coverage, maximum rule length, beam width) can be easily customized to balance accuracy and interpretability. In our implementation, we set the minimum coverage to 5 instances and restrict rule length to a maximum of 3 conditions to avoid overfitting. RuleKit's flexible configuration enables reproducible rule extraction while allowing adaptation to different datasets and label distributions.

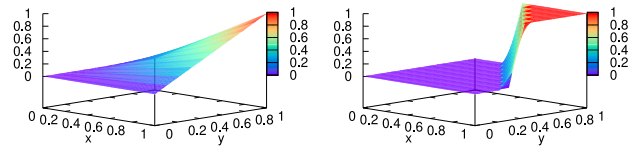


Fig. 2. (left) Product t -norm. (right) LNN- $\wedge$ ($\alpha = 0.7$).

3.2. Logical neural networks

Recently, the growing complexity of deep learning has driven interest in developing reasoning models (Guidotti et al., 2018). While progress has been made, leveraging rules in first-order logic (FOL) provides a far more powerful framework for reasoning tasks. In neuro-symbolic AI, the learning process often substitutes conjunctions (AND operations) and disjunctions (OR operations) with differentiable t -norms and t -conorms, respectively. However, these norms lack learnable parameters, causing their behavior to remain static. This limitation reduces their flexibility and restricts their ability to accurately model the underlying data, hindering the effectiveness of these reasoning models.

Logical Neural Networks (LNNs) (Riegel et al., 2020) introduce parameters to operators, enabling more effective data-driven learning while preserving the precise semantics of first-order logic (FOL). LNNs achieve this by imposing constraints during the learning of operators such as conjunction ($\wedge$). The formulation for LNN- $\wedge$ is given by:

$$\max(0, \min(1, \beta - \omega_1(1-x) - \omega_2(1-y))) \quad (6)$$

$$\text{subject to : } \beta - (1-\alpha)(\omega_1 + \omega_2) \geq \alpha \quad (6a)$$

$$\beta - \alpha\omega_1 \leq 1 - \alpha \quad (6b)$$

$$\beta - \alpha\omega_2 \leq 1 - \alpha \quad (6c)$$

$$\omega_1, \omega_2 \geq 0$$

where $x, y \in [0, 1]$ are inputs, $\alpha \in [\frac{1}{2}, 1]$ is a hyperparameter and $\beta, \omega_1, \omega_2$ are learnable parameters. Note that $\max(0, \min(1, \cdot))$ clamps the output of LNN- $\wedge$ between 0 and 1 regardless of $\beta, \omega_1, \omega_2, x$ and y . The constraints are most noteworthy. While Boolean conjunction only returns 1 or *TRUE* when both inputs are 1, LNNs relax this condition by using α as a proxy for 1 (conversely, $1-\alpha$ as a proxy for 0). Specifically, Eq. (6a) ensures that the output from the LNN exceeds the value α provided that its input also exceeds α . Similarly, Eqs. (6b) and (6c) limit the inputs to the LNN to one high and one low. For instance, Constraint (6b) forces the output of LNN- $\wedge$ to be less than $1-\alpha$ for $y=1$ and $x \leq 1-\alpha$. This formulation allows for unconstrained learning when $x, y \in [1-\alpha, 1]$. By adjusting the value of α , users can manipulate the extent of learning; increasing α expands the area of unrestricted learning, while decreasing it narrows this region. Fig. 2 depicts product t -norm and LNN- $\wedge$ ($\alpha = 0.7$).

In case the application requires even more degrees of freedom, it can be achieved through the inclusion of slacks:

$$\max(0, \min(1, \beta - \omega_1(1-x) - \omega_2(1-y)))$$

subject to :

$$\beta - (1-\alpha)(\omega_1 + \omega_2) + \Delta \geq \alpha \quad (7)$$

$$\beta - \alpha\omega_1 \leq 1 - \alpha + \delta_1$$

$$\beta - \alpha\omega_2 \leq 1 - \alpha + \delta_2$$

$$\omega_1, \omega_2, \delta_1, \delta_2, \Delta \geq 0$$

where δ_1 , δ_2 , and Δ denote slack variables. When the constraints in (6a) (6b) (6c) are not met, slacks are employed to adjust the inequality's direction, alleviating the training pressure on parameters ω_1, ω_2 and β . LNN negation functions as a pass-through operator, represented by

¹ RuleKit: <https://adaa-polis.github.io/RuleKit-python/serve/index.html>

Table 2
Logic Rules Based on Software Metrics.

Rule	Content	Label	Explanation
Rule1	IF SumCyclomaticStrict $\in (12.0, \infty)$	1	High control flow complexity; harder to test and verify; prone to logic bugs
Rule2	IF MaxFanIn $\in (15.0, \infty)$	1	High input coupling; likely a central component, vulnerable to widespread impact
Rule3	IF SumCyclomaticStrict $\in (12.0, \infty)$ AND MaxFanIn $\in (15.0, \infty)$	1	Combination of Rule1 and Rule2; highly complex and coupled $\rightarrow$ high vulnerability risk
Rule4	IF CommentDensity $\in (5.0, \infty)$	0	Well-documented function; often maintained carefully; low risk
Rule5	IF MaxFanIn $\in (15.0, \infty)$ AND CommentDensity $\in (5.0, \infty)$ AND SumFanOut $\in (0.0, 5.0)$	1	Highly coupled and well-documented with low outward calls; may hide unchecked logic
Rule6	IF MaxFanIn $\in (15.0, \infty)$ AND CommentDensity $\in (5.0, \infty)$ AND SumFanOut $\in (10.0, \infty)$	0	Similar to Rule5, but outward calls suggest coordination function; less risky
Rule7	IF CommentDensity $\in (5.0, \infty)$ AND SumFanOut $\in (0.0, 5.0)$	1	Highly documented but limited output; may contain hidden risky logic
Rule8	IF SumEssential $\in (25.0, \infty)$ AND CountLinePreprocessor $\in (20.0, \infty)$ AND CommentDensity $\in (-\infty, 0.5)$	1	High complexity, heavy macro usage, and poor documentation $\rightarrow$ high-risk code
Rule9	IF MaxMaxNesting $\in (-\infty, 2.0)$ AND SumFanIn $\in (-\infty, 5.0)$	0	Low nesting and low coupling suggest modular and safe structure
Rule10	...	...	...

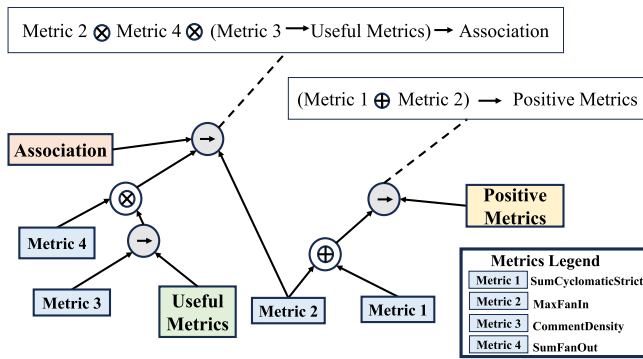


Fig. 3. Process of reasoning.

$LNN-\neg(x) = 1 - x$, while LNN disjunction is defined in relation to $LNN-\wedge$:

$$LNN-\vee(x, y) = 1 - LNN-\wedge(1 - x, 1 - y) \quad (8)$$

Traditional backpropagation struggles to address linear inequality constraints like Constraint (6a). In contrast, the Logical Neural Network (LNN) framework incorporates dedicated learning algorithms specifically designed to overcome this limitation. Comprehensive details about the LNN framework and its methodologies are provided in the work of Riegel et al. (2020).

3.3. Simple example

The architecture of the LNN model is depicted in Fig. 4. To show how reasoning works, here's a simple example. This example was generated using the REVEAL (Chakraborty et al., 2021) dataset, and similar examples can be created for other datasets. Initially, we generated several rules using the rule learning approach, as detailed in Table 2. Within these rules, a label of 1 indicates vulnerable software, whereas a label of 0 denotes a non-vulnerable project. The last column represents the meaning of the logic rules. Moreover, vulnerable samples are termed as positive samples. At the same time, LNN model is employed to construct the predicates required for reasoning. There are three types of relationships for software metrics: *Association*, *Useful Metrics* and *Positive Metrics*. The assumption suggests that useful metrics include both positive and negative types, and metrics associated with positive

types are also likely to be positive. *Association* represents the presence of a connection between two metrics. For instance, the repeated appearance of a metric combination across various rules suggests a connection between these metrics. *Useful Metrics* represents metrics distinctly linked to labels, including both positive and negative labels. *Positive Metrics* denotes the metric directly linked to the positive label, which is the centerpiece of our analysis. Our goal is to identify all metrics with a significant correlation to vulnerable samples.

The reasoning process is more clearly depicted in Fig. 3, where “ $\oplus$ ” represents “or”, and “ $\otimes$ ” represents “and”. Rule 1, Rule 2, and Rule 4 constitute very simple logical propositions, each characterized by one single antecedent. This implies that the metric present in the antecedent of these rules must be considered as *Useful Metrics*. Since only one “antecedent” follows “IF”, it indicates that this singular metric will determine the target's classification as a vulnerable sample or not. Based on Rule 1 and Rule 2, it can be easily inferred that either SumCyclomaticStrict (metric1) or MaxFanIn (metric2) is associated with the vulnerable sample. Rule 3 indicates that the combination of two positive metrics also results in a positive metric. Additionally, CommentDensity (metric3) is associated with negative samples based on Rule 4. Based on Rules 5 and 6, MaxFanIn (metric2), CommentDensity (metric3), and SumFanOut (metric4) appear in multiple rules, suggesting an association relationship among them. We assume that metrics associated with positive metrics are highly likely to be positive themselves. In conclusion, given the association between software metrics and Rule 7, combined with the negative metric of CommentDensity (metric3), it is inferred that SumFanOut (metric4) possesses a positive metric. Since if both CommentDensity (metric3) and SumFanOut (metric4) are negative metrics, Rule 6 cannot have a label of 1, thus SumFanOut (metric4) must be a positive metric. This is the process of reasoning whether SumFanOut (metric4) is a positive or negative metric.

In summary, the reasoning process organizes metrics into three categories: *Positive Metrics*, directly associated with vulnerable samples; *Negative Metrics*, directly linked to non-vulnerable samples; and *Useful Metrics*, which appear in the rules without direct classification. Metrics that occur in multiple rules are categorized as *Association*. Furthermore, metrics connected to positive metrics are also treated as positive, facilitating the identification of all positive metrics in the dataset. These identified positive metrics are then exclusively used as attributes for prediction within the deep neural network. This methodology seamlessly integrates the LRNN layer with the neural network classifier.

```

*****
AXIOM Implies: (Association(0, 1) → ((Useful_Metrics(0) → Useful_Metrics(1)) ∧ (Useful_Metrics(1) → Useful_Metrics(0))))
OPEN If: ((Useful_Metrics(0) → Useful_Metrics(1)) ∧ (Useful_Metrics(1) → Useful_Metrics(0)))
OPEN Implies: (Useful_Metrics(0) → Useful_Metrics(1))
OPEN Predicate: Association
AXIOM Implies: (Useful_Metrics(0) → Positive_Metrics(0))
OPEN Predicate: Positive_Metrics
OPEN Predicate: Useful_Metrics
*****

```

Fig. 4. Structure of LNN Model.

The above provides a basic illustration of the reasoning process through an example; however, in practice, most rules, such as Rule 8 and Rule 9, are lengthy and intricate. To handle this complexity, the entire reasoning process leverages the Python API for Logical Neural Networks (LNN)² developed by Riegel et al. (2020).

4. Experiments

This section presents experiments conducted to validate the effectiveness of the MC-LRNN in vulnerability prediction. It begins with a description of the datasets and the baseline methodology, followed by an overview of the evaluation metrics. Finally, the performance of the proposed MC-LRNN approaches in predicting vulnerabilities is assessed.

4.1. Datasets

We selected three widely used vulnerability datasets to evaluate our method: **REVEAL** (Chakraborty et al., 2021), **SATE IV Juliet** (Okun et al., 2013) and **SARD** (Black et al., 2017). Their details are as follows:

- **REVEAL Dataset:** The REVEAL dataset is a real-world vulnerability dataset constructed from open-source C projects, aiming to support vulnerability prediction at the function level. It contains 18,169 C functions, among which 1,664 are labeled as vulnerable based on cross-referencing with CVE reports and manual inspection. The remaining 16,505 functions are labeled as non-vulnerable. Due to the real-world nature of the dataset, it exhibits a highly **imbalanced class distribution**, with vulnerable functions comprising only approximately 9.16% of the total.
- **Juliet Dataset:** The Juliet Test Suite comprises 117,728 manually generated C/C++ functions, with 76,567 functions containing vulnerabilities across 116 vulnerability types. In this study, the dataset is divided into vulnerable and non-vulnerable functions without differentiating specific vulnerability types. As a result, the dataset is treated as a **binary classification** problem.
- **SARD Dataset:** The Software Assurance Reference Dataset (SARD) is a widely used public resource for software vulnerability testing and analysis. It currently contains 177,184 entries with test cases. Code samples in SARD are labeled as Good (no vulnerabilities), Bad (containing vulnerabilities), or Mixed (including both vulnerabilities and their corresponding patches). Consequently, this dataset represents a **multi-class classification** problem.

Table 3

Descriptive statistics of selected metrics in the Juliet dataset.

Metric	Class	Mean	Median	Max	Min
CountLineCode	Vulnerable	42.1	40	120	5
	Clean	38.3	37	110	4
SumCyclomaticStrict	Vulnerable	5.8	5	18	1
	Clean	4.9	4	16	1
MaxMaxNesting	Vulnerable	3.7	3	12	0
	Clean	2.9	2	10	0
SumFanOut	Vulnerable	6.2	5	21	0
	Clean	4.6	4	18	0

4.1.1. Data pre-processing

Specialized software metrics for vulnerability prediction have been proposed in prior work (Shin et al., 2010). In this study, we adopt a suite of static software metrics that are widely used in vulnerability prediction tasks. These metrics quantify various properties of code, including its size, structural complexity, and coupling characteristics, at both the function and file levels. A complete list of the selected metrics and their definitions is provided in Table 1.

To compute these metrics, we employed Lizard, an open-source static analysis tool that supports multiple programming languages, including C, C++, and Java. Lizard extracts fine-grained complexity metrics at the function level, such as cyclomatic complexity, maximum nesting depth, and number of parameters. Its lightweight design and tolerance for incomplete or non-compilable code make it particularly suitable for analyzing vulnerability datasets such as Juliet, SARD, and REVEAL, which often contain isolated functions, code fragments, or patched code.

In addition to automated extraction using Lizard, we incorporated ChatGPT-4 to assist with preliminary code structure analysis and the interpretation of extracted metrics. All metric results were subsequently validated through manual inspection to ensure correctness and semantic consistency. This hybrid workflow enabled us to efficiently and reliably annotate large-scale vulnerability datasets with structural metrics for downstream machine learning applications.

To gain a comprehensive understanding of the distribution of structural properties in the datasets, we computed descriptive statistics (mean, median, maximum, and minimum) for the selected metrics, separately for vulnerable and clean samples. These statistics provide insight into the inherent complexity differences between vulnerable and non-vulnerable code and help explain the learning difficulty faced by classifiers.

Table 3 summarizes the statistics for the Juliet dataset. Vulnerable functions tend to have slightly higher values for metrics such as cyclomatic complexity, nesting depth, and fan-out, suggesting increased structural complexity. Table 4 presents corresponding statistics for the REVEAL dataset, where the differences between vulnerable and clean functions are more pronounced due to the real-world nature of the code. The SARD dataset shows similar trends but with greater variation across samples, as shown in Table 5.

² LNN: <https://ibm.github.io/LNN/introduction.html>

Table 4
Descriptive statistics of selected metrics in the REVEAL dataset.

Metric	Class	Mean	Median	Max	Min
CountLineCode	Vulnerable	51.6	48	174	7
	Clean	42.3	40	155	5
SumCyclomaticStrict	Vulnerable	6.8	6	21	1
	Clean	4.9	4	17	1
MaxMaxNesting	Vulnerable	4.3	4	10	0
	Clean	2.9	2	8	0
SumFanOut	Vulnerable	7.1	6	22	0
	Clean	4.8	4	17	0

Table 5
Descriptive statistics of selected metrics in the SARD dataset.

Metric	Class	Mean	Median	Max	Min
CountLineCode	Vulnerable	48.2	45	135	7
	Clean	39.7	37	121	5
SumCyclomaticStrict	Vulnerable	6.5	6	21	1
	Clean	4.7	4	17	1
MaxMaxNesting	Vulnerable	4.1	4	10	0
	Clean	2.8	2	8	0
SumFanOut	Vulnerable	6.9	6	22	0
	Clean	4.5	4	16	0

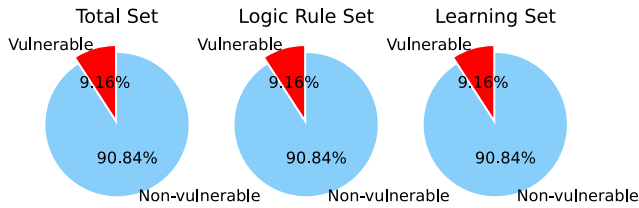


Fig. 5. Partitioning of the REVEAL Dataset.

Overall, these statistics confirm prior findings that vulnerable code is generally more complex and harder to maintain than clean code (Chowdhury and Zulkernine, 2011).

4.1.2. Dataset segmentation

The complete dataset is randomly and equally divided into two parts: the *logic rule set* and the *learning set*. The logic rule set is used to derive logical rules, while the learning set is reserved for training and testing machine learning models. For instance, in the REVEAL dataset (total 18,169 samples), 9,085 samples are used for logic rule construction, and the remaining 9,084 samples are assigned to the learning set. Among them, the class distribution is preserved during the split (i.e., approximately 9.16% of samples in each subset are vulnerable) in Fig. 5. Similarly, for Juliet (total 117,728 samples) and SARD (total 177,184 samples), we randomly split each dataset into two equal parts. Table 6 reports the exact number of samples in each subset for all datasets.

The learning set is further split into 70% training and 30% testing sets. The training set is used to optimize the parameters of the deep learning models, while the testing set is used to evaluate the performance of MC-LRNN and other baselines. All results are averaged over five random runs to ensure statistical stability.

4.2. Baseline

We employed several widely used AI algorithms as baseline methods and conducted comprehensive comparisons with our proposed MC-LRNN model under identical environmental and data preprocessing conditions. Among them, we adopted CodeBERT, a state-of-the-art pretrained large language model specifically designed for source code, as our primary baseline. In addition, we evaluated a rule-based prediction approach, where predictions are generated directly from learned

logic rules. To further examine the feature selection capabilities of MC-LRNN, we incorporated a neural network that leverages Principal Component Analysis (PCA) for feature extraction, along with two commonly used feature selection models: RFE-RF (Recursive Feature Elimination with Random Forest) and L1-LR (L1-regularized Logistic Regression).

1. **CodeBERT**: We used the pretrained CodeBERT model, a transformer-based model trained on source code corpora. The model receives raw source code as input and generates contextual embeddings. We added a classification head on top of the [CLS] token and fine-tuned the model for binary vulnerability prediction (Feng et al., 2020a).
2. **Rule Learning**: Predictions on the test set are directly derived from the generated rules (Gudyś et al., 2024).
3. **PCA-NN**: Before training the deep neural network, the Principal Component Analysis (PCA) algorithm was employed for feature preprocessing (Abdi and Williams, 2010).
4. **RFE-RF**: Recursive Feature Elimination (RFE) was applied in conjunction with a Random Forest classifier to iteratively eliminate the least important features. The selected subset was then used to train a neural network for vulnerability prediction (Guyon et al., 2002).
5. **L1-LR**: L1-regularized Logistic Regression was used to perform embedded feature selection, where features with zero weights were eliminated. The remaining features were subsequently input to a neural network model for final prediction (Ng, 2004).

4.3. Evaluation metrics

To comprehensively evaluate the effectiveness of the MC-LRNN model, widely recognized metrics such as Precision (P), Recall (R), Accuracy (A), and F-measure (F) are employed (Sasaki et al., 2007). The distinctions between Precision, Recall, and Accuracy are elaborated in Streiner and Norman (2006) and Buckland and Gey (1994). These metrics are commonly used in vulnerability prediction to effectively measure and compare model performance.

4.3.1. Evaluation metrics for binary classification

Prediction performance evaluation often involves analyzing data through a confusion matrix, which summarizes the classification outcomes of a prediction model across various vulnerability categories, compared to their actual classifications. Table 7 presents a confusion matrix with four possible outcomes for vulnerability prediction: True Positive (TP): The number of vulnerable instances correctly classified as vulnerable. False Negative (FN): The number of vulnerable instances incorrectly classified as non-vulnerable. False Positive (FP): The number of non-vulnerable instances incorrectly classified as vulnerable. True Negative (TN): The number of non-vulnerable instances correctly classified as non-vulnerable. Using the confusion matrix, several standard performance evaluation measures widely used in vulnerability prediction studies can be defined by the following formulas:

$$\text{Precision} = \frac{TP}{TP + FP} \quad (9)$$

$$\text{Recall} = \frac{TP}{TP + FN} \quad (10)$$

$$\text{F-Measure} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \quad (11)$$

$$\text{Accuracy} = \frac{TP + TN}{TP + FN + FP + TN} \quad (12)$$

Table 6

Sample counts for logic rule and learning sets.

Dataset	Total Samples	Logic Rule Set	Learning Set
REVEAL	18,169	9,085	9,084
Juliet	117,728	58,864	58,864
SARD	177,184	88,592	88,592

Table 7

Confusion Matrix for Binary Classification.

	Predicted vulnerable	Predicted non-vulnerable
Actual vulnerable	TP	FN
Actual non-vulnerable	FP	TN

Table 8

Confusion Matrix for Multi-class Classification.

	Predicted A	Predicted B	Predicted C
Actual A	A $\rightarrow$ A	A $\rightarrow$ B	A $\rightarrow$ C
Actual B	B $\rightarrow$ A	B $\rightarrow$ B	B $\rightarrow$ C
Actual C	C $\rightarrow$ A	C $\rightarrow$ B	C $\rightarrow$ C

Table 9

Results on the Juliet Dataset (Binary Classification)

Method	Precision	Recall	F-measure	Accuracy
CodeBERT	0.8031	0.8327	0.8176	0.7928
Rule Learning	0.7748	0.7486	0.7614	0.7421
PCA-NN	0.7929	0.7826	0.7854	0.7654
RFE-RF	0.7764	0.7653	0.7702	0.7602
L1-LR	0.7639	0.7487	0.7562	0.7511
MC-LRNN	0.8005	0.8405	0.8184	0.7963

4.3.2. Evaluation metrics for multi-class classification

The confusion matrix for multi-class classification is similar to that of binary classification but extends to higher dimensions. For instance, in three dimensions, refer to Table 8. Using category A as an example, TP_A includes A $\rightarrow$ A; FN_A comprises A $\rightarrow$ B and A $\rightarrow$ C; FP_A consists of B $\rightarrow$ A and C $\rightarrow$ A; and TN_A includes B $\rightarrow$ B and C $\rightarrow$ C. Applying the same approach to categories B and C, the formulas for the four performance metrics are as follows:

$$\text{Precision}_A = \frac{TP_A}{TP_A + FP_{B \rightarrow A} + FP_{C \rightarrow A}} \quad (13)$$

$$\text{Recall}_A = \frac{TP_A}{TP_A + FN_{A \rightarrow B} + FN_{A \rightarrow C}} \quad (14)$$

$$\text{F-Measure}_A = 2 \times \frac{\text{Precision}_A \times \text{Recall}_A}{\text{Precision}_A + \text{Recall}_A} \quad (15)$$

$$\text{Accuracy}_A = \frac{TP_A + TN_{B \rightarrow B} + TN_{C \rightarrow C}}{\text{Total number of samples}} \quad (16)$$

‘A’ refers to the label ‘Good’, ‘B’ to ‘Bad’, and ‘C’ to ‘Mixed’ on the SARD Dataset.

In the presence of class imbalance, Accuracy can be misleading, as a model may achieve high accuracy by simply ignoring the minority class. In contrast, Precision and Recall provide better insights into the model’s performance on the minority class, with Recall particularly useful for identifying potential under-detection issues. The F1-score, which balances Precision and Recall, is more suitable for evaluating models on imbalanced datasets.

4.4. Results

To validate the efficacy of our approach, we compare it with state-of-the-art deep learning algorithms. Experimental results, presented in Tables 9, 10, 11, 12 and 13 demonstrate the performance of our method across three datasets. A comprehensive evaluation against baseline methods confirms the effectiveness of the MC-LRNN model in vulnerability prediction tasks.

Table 10

Results on the SARD Dataset (Multi-class Classification)

Method	Precision_A	Recall_A	F-measure_A	Accuracy
CodeBERT	0.372	0.361	0.271	0.336
Rule Learning	0.318	0.329	0.250	0.319
PCA-NN	0.335	0.316	0.213	0.329
RFE-RF	0.292	0.281	0.198	0.301
L1-LR	0.268	0.290	0.204	0.294
MC-LRNN	0.360	0.368	0.279	0.342

	Precision_B	Recall_B	F-measure_B	
CodeBERT	0.346	0.337	0.243	
Rule Learning	0.275	0.411	0.240	
PCA-NN	0.334	0.365	0.244	
RFE-RF	0.295	0.309	0.220	
L1-LR	0.261	0.280	0.198	
MC-LRNN	0.338	0.323	0.220	

	Precision_C	Recall_C	F-measure_C	
CodeBERT	0.432	0.321	0.320	
Rule Learning	0.300	0.216	0.151	
PCA-NN	0.317	0.306	0.194	
RFE-RF	0.284	0.277	0.190	
L1-LR	0.265	0.268	0.179	
MC-LRNN	0.465	0.334	0.343	

Table 11

Results on the REVEAL Dataset (Binary Classification with Class Imbalance)

Method	Precision	Recall	F-measure	Accuracy
CodeBERT	0.7821	0.3245	0.4590	0.9054
Rule Learning	0.7156	0.2783	0.3998	0.8912
PCA-NN	0.7267	0.3021	0.4261	0.8975
RFE-RF	0.7419	0.2504	0.3719	0.8937
L1-LR	0.6923	0.2317	0.3451	0.8881
MC-LRNN	0.7684	0.4867	0.5925	0.9103

4.4.1. Results for within-project prediction

We adopt within-project prediction, where the dataset is partitioned as illustrated in Fig. 1, and the same dataset is used for both training and prediction. Tables 9, 10, and 11 present a detailed numerical comparison of the MC-LRNN model and baseline models across Juliet, SARD and REVEAL datasets. Each column corresponds to a specific performance metric, including Precision, Recall, Accuracy, and F-measure, with the highest scores for each metric highlighted in bold. The values represent averages derived from multiple experimental runs, ensuring reliability and representativeness.

As shown in Table 9 for binary classification tasks, MC-LRNN achieves performance comparable to CodeBERT—the state-of-the-art LLM-based pre-training model—and outperforms other baseline methods. For multiclass classification (Table 10), MC-LRNN performs similarly to CodeBERT, but due to the increased complexity of multiclass problems, MC-LRNN slightly surpasses CodeBERT on most evaluation metrics. Notably, in real-world binary classification tasks with class imbalance (Table 11), MC-LRNN demonstrates significantly better performance than both CodeBERT and other baselines.

Based on the above experimental results, the MC-LRNN model has achieved state-of-the-art (SOTA) performance, particularly in addressing the real-world issue of class imbalance—a common characteristic in software vulnerability prediction datasets.

Table 12

Cross-Project Results: Train on Juliet, Test on SARD (Binary Classification)

Method	Precision	Recall	F-measure	Accuracy
CodeBERT	0.6214	0.2983	0.4021	0.7352
Rule Learning	0.5841	0.2625	0.3621	0.7127
PCA-NN	0.5987	0.2759	0.3748	0.7193
RFE-RF	0.5634	0.2441	0.3395	0.7014
L1-LR	0.5389	0.2205	0.3124	0.6930
MC-LRNN	0.6087	0.3746	0.4625	0.7481

Table 13

Cross-Project Results: Train on SARD, Test on Juliet (Binary Classification)

Method	Precision	Recall	F-measure	Accuracy
CodeBERT	0.6935	0.4032	0.5085	0.7529
Rule Learning	0.6419	0.3687	0.4664	0.7341
PCA-NN	0.6581	0.3795	0.4796	0.7395
RFE-RF	0.6293	0.3441	0.4452	0.7280
L1-LR	0.6107	0.3269	0.4251	0.7214
MC-LRNN	0.6784	0.4557	0.5453	0.7654

4.4.2. Results for cross-project prediction

We also considered cross-project prediction, where the model is trained on one dataset and tested on another. To ensure label consistency, we treated both the Juliet and SARD datasets as binary classification problems. The results are presented in Tables 12 and 13.

From these two tables, it can be observed that MC-LRNN outperforms other baselines, including the LLMs-based CodeBERT, demonstrating that our proposed model is not only effective in handling class imbalance but also capable of cross-project prediction, indicating strong generalizability.

5. Discussion

5.1. Model advantages and comparative performance

This paper introduces a novel algorithm that establishes a new benchmark on the Juliet, SARD and REVEAL datasets, outperforming models such as LLMs-based model, rule learning-based method, neural networks with PCA and other feature selection models in terms of performance. Compared to LLMs-based architectures, MC-LRNN—built on the Logical Neural Network (LNN) framework—introduces reasoning capabilities to analyze relationships among features and perform feature selection, providing a novel approach to this task. This is particularly advantageous when working with real-world datasets characterized by imbalanced class distributions.

Unlike traditional rule learning methods, the MC-LRNN approach incorporates generated rules as a penalty term within the neural network's attributes, rather than directly using the rules for predictions. By leveraging the computational power of neural networks, this method enhances prediction accuracy. Additionally, compared to traditional feature extraction or selection models such as PCA, RFE-RF, and L1-LR, MC-LRNN consistently outperforms them with significantly better performance.

In summary, MC-LRNN emerges as an effective and innovative solution for automatic vulnerability prediction (AVP), combining the strengths of neural networks and logical reasoning to achieve superior performance and feature selection.

5.2. Threats to validity

The following points outline key risks that may affect the validity of our experiments:

- **Bias of dataset.** Dataset bias presents a significant challenge, as a single prediction model may demonstrate inconsistent performance across different code projects, reflecting a lack of uniformity in diverse environments. To address this issue, the MC-LRNN model was evaluated using projects from three distinct datasets: Juliet, SARD and REVEAL. The consistent results across multiple experiments conducted on various code projects within these datasets provide strong evidence of the model's reliability and effectiveness, alleviating concerns about dataset bias and reinforcing its generalizability.
- **Bias of evaluation measures.** Relying solely on evaluation metrics such as Recall and Accuracy to evaluate vulnerability prediction performance can lead to biased conclusions, as these metrics fail to provide a complete picture of the model's effectiveness. For example, Accuracy may disregard imbalanced datasets, while Recall alone does not account for false positives. Evaluation metrics like the F-measure, which balances Precision and Recall, offer a more comprehensive and equitable assessment of model performance. To address this limitation, this study employs a diverse set of evaluation metrics—Precision, Recall, Accuracy, and F-measure—ensuring a thorough and reliable empirical evaluation of vulnerability prediction performance.
- **Comparison accuracy.** Many authors of related works do not make the source code for their methods publicly available, which can compromise the precision and reliability of comparisons. To overcome this challenge, we carefully re-implemented these methods based on the detailed descriptions provided in their respective papers. This approach ensures that the comparisons are accurate and fair, despite the absence of access to the original implementations.

6. Conclusion

This paper introduces the Multi-Class Logic Rules Neural Network (MC-LRNN), a novel system for automatic vulnerability prediction (AVP). Experimental results demonstrate that MC-LRNN outperforms major related models when evaluated on the Juliet, SARD and REVEAL datasets. The MC-LRNN model is capable of handling both binary and multi-class classification tasks, while also effectively mitigating the impact of class imbalance in real-world scenarios. The results indicate that MC-LRNN can infer software metrics associated with vulnerable samples through first-order logic rules. Utilizing these inferred metrics as model attributes introduces a novel approach to feature engineering, offering an effective method for feature selection.

Future Work: To enhance the applicability and robustness of the MC-LRNN model, future research will focus on validating its performance across different programming languages, including C[‡], C++, and Python. Another key direction involves converting the first-order logic rules in MC-LRNN into probabilistic logic programming to improve its reasoning capabilities and adaptability for complex predictive tasks.

CRedit authorship contribution statement

Yuxiang Shang: Writing – original draft, Visualization, Software, Methodology, Data curation. **Shaoying Liu:** Writing – review & editing, Supervision, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgment

This work was supported by the Japan Science and Technology Agency (JST), Next Generation Researcher Program.

Data availability

Data will be made available on request.

References

- Abdi, H., Williams, L.J., 2010. Principal component analysis. *Wiley Interdiscip. Rev.: Comput. Stat.* 2 (4), 433–459.
- Black, P.E., et al., 2017. SARD: A software assurance reference dataset. In: Anonymous Cybersecurity Innovation Forum, vol. 85.
- Buckland, M., Gey, F., 1994. The relationship between recall and precision. *J. Am. Soc. Inf. Sci.* 45 (1), 12–19.
- Chakraborty, S., Krishna, R., Ding, Y., Ray, B., 2021. Deep learning based vulnerability detection: Are we there yet? *IEEE Trans. Softw. Eng.* 48 (9), 3280–3296.
- Chen, M., Tworek, J., Jun, H., Yuan, Q., De Oliveira Pinto, H.P., Kaplan, J., Edwards, H., Burda, Y., Joseph, N., Brockman, G., et al., 2021. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*.
- Chen, X.-w., Wasikowski, M., 2008. FAST: A ROC-based feature selection metric for small samples and imbalanced data classification problems. In: *Proc. of the 14th ACM SIGKDD Int. Conf. on Knowledge Discovery and Data Mining*. pp. 124–132.
- Chowdhury, I., Zulkernine, M., 2011. Using complexity, coupling, and cohesion metrics as early indicators of vulnerabilities. *J. Syst. Archit.* 57 (3), 294–313.
- Church, K.W., 2017. Word2Vec. *Nat. Lang. Eng.* 23 (1), 155–162.
- Cropper, A., Dumančić, S., 2022. Inductive logic programming at 30: a new introduction. *J. Artificial Intelligence Res.* 74, 765–850.
- Dash, T., Chitlangia, S., Ahuja, A., Srinivasan, A., 2022. A review of some techniques for inclusion of domain-knowledge into deep neural networks. *Sci. Rep.* 12 (1), 1040.
- Duan, X., Wu, J., Ji, S., Rui, Z., Luo, T., Yang, M., Wu, Y., 2019. VulSniper: Focus your attention to shoot fine-grained vulnerabilities. In: *Proc. of the Int. Joint Conf. on Artificial Intelligence. IJCAI*, pp. 4665–4671.
- Fagin, R., Riegel, R., Gray, A., 2020. Foundations of reasoning with uncertainty via real-valued logics. *arXiv preprint arXiv:2008.02429*.
- Feng, H., Fu, X., Sun, H., Wang, H., Zhang, Y., 2020b. Efficient vulnerability detection based on abstract syntax tree and deep learning. In: *IEEE INFOCOM 2020 - IEEE Conf. on Computer Communications Workshops. INFOCOM WKSHPS*, pp. 722–727.
- Feng, Z., Guo, D., Tang, D., Duan, N., Feng, X., Gong, M., Shou, L., Qin, B., Liu, T., Jiang, D., et al., 2020a. CodeBERT: A pre-trained model for programming and natural languages. *arXiv preprint arXiv:2002.08155*.
- Fürnkranz, J., 2002. A pathology of bottom-up hill-climbing in inductive rule learning. In: *Int. Conf. on Algorithmic Learning Theory*. Springer, pp. 263–277.
- Fürnkranz, J., Gamberger, D., Lavrač, N., 2012. *Foundations of Rule Learning*. Springer Science & Business Media.
- Fürnkranz, J., Klieger, T., 2015. A brief overview of rule learning. In: *Int. Symp. on Rules and Rule Markup Languages for the Semantic Web*. Springer, pp. 54–69.
- Fürnkranz, J., Widmer, G., 1994. Incremental reduced error pruning. In: *Machine Learning Proceedings 1994*. Elsevier, pp. 70–77.
- Gegick, M., Williams, L., Osborne, J., Vouk, M., 2008. Prioritizing software security fortification through code-level metrics. In: *Proc. of the 4th ACM Workshop on Quality of Protection*. pp. 31–38.
- Gudyś, A., Sikora, M., Wróbel, Ł., 2020. RuleKit: A comprehensive suite for rule-based learning. *Knowl.-Based Syst.* 194, 105480.
- Gudyś, A., Sikora, M., Wróbel, Ł., 2024. Separate and conquer heuristic allows robust mining of contrast sets in classification, regression, and survival data. *Expert Syst. Appl.* 123376.
- Guidotti, R., Monreale, A., Ruggieri, S., Turini, F., Giannotti, F., Pedreschi, D., 2018. A survey of methods for explaining black box models. *ACM Comput. Surv.* 51 (5), 1–42.
- Guo, J., Wang, Z., Li, H., Xue, Y., 2023. Detecting vulnerability in source code using CNN and LSTM network. *Soft Comput.* 27 (2), 1131–1141.
- Guyon, I., Weston, J., Barnhill, S., Vapnik, V., 2002. Gene selection for cancer classification using support vector machines. *Mach. Learn.* 46 (1), 389–422.
- Hitzler, P., Sarker, M.K., 2022. *Neuro-Symbolic Artificial Intelligence: The State of the Art*. IOS Press.
- Hu, Z., Ma, X., Liu, Z., Hovy, E., Xing, E., 2016. Harnessing deep neural networks with logic rules. *arXiv preprint arXiv:1603.06318*.
- Kaloutsoglou, I., Siavvas, M., Ampatzoglou, A., Kehagias, D., Chatzigeorgiou, A., 2023. Software vulnerability prediction: A systematic mapping study. *Inf. Softw. Technol.* 164, 107303.
- Kaloutsoglou, I., Siavvas, M., Ampatzoglou, A., Kehagias, D., Chatzigeorgiou, A., 2024. Vulnerability prediction using pre-trained models: An empirical evaluation. In: *Proc. of the 32nd Int. Conf. on Modeling, Analysis and Simulation of Computer and Telecommunication Systems. MASCOTS, IEEE*, pp. 1–6.
- Kamalov, F., Thabtah, F., Leung, H.H., 2023. Feature selection in imbalanced data. *Ann. Data Sci.* 10 (6), 1527–1541.
- Li, T., Srikumar, V., 2019. Augmenting neural networks with first-order logic. *arXiv preprint arXiv:1906.06298*.
- Li, Z., Zou, D., Xu, S., Jin, H., Zhu, Y., Chen, Z., 2021. Sysevr: A framework for using deep learning to detect software vulnerabilities. *IEEE Trans. Dependable Secur. Comput.* 19 (4), 2244–2258.
- Li, Z., Zou, D., Xu, S., Ou, X., Jin, H., Wang, S., Deng, Z., Zhong, Y., 2018. Vuldeepecker: A deep learning-based system for vulnerability detection. *arXiv preprint arXiv:1801.01681*.
- Lin, W., Cai, S., 2021. An empirical study on vulnerability detection for source code software based on deep learning. In: *Proc. of the 21st IEEE Int. Conf. on Software Quality, Reliability and Security Companion. QRS-C, IEEE*, pp. 1159–1160.
- Lin, G., Zhang, J., Luo, W., Pan, L., Xiang, Y., 2017. POSTER: Vulnerability discovery with function representation learning from unlabeled projects. In: *Proc. of the 2017 ACM SIGSAC Conf. on Computer and Communications Security*. pp. 2539–2541.
- Liu, H., Zhou, M., Liu, Q., 2019. An embedded feature selection method for imbalanced data classification. *IEEE/CAA J. Autom. Sin.* 6 (3), 703–715.
- Maldonado, S., López, J., 2018. Dealing with high-dimensional class-imbalanced datasets: Embedded feature selection for SVM classification. *Appl. Soft Comput.* 67, 94–105.
- Maldonado, S., Weber, R., Famili, F., 2014. Feature selection for high-dimensional class-imbalanced data sets using support vector machines. *Inform. Sci.* 286, 228–246.
- Matharaarachchi, S., Domaratzki, M., Muthukumarana, S., 2021. Assessing feature selection method performance with class imbalance data. *Mach. Learn. Appl.* 6, 100170.
- Meneely, A., Williams, L., 2009. Secure open source collaboration: an empirical study of linus' law. In: *Proc. of the 16th ACM Conf. on Computer and Communications Security*. pp. 453–462.
- Muggleton, S., De Raedt, L., 1994. Inductive logic programming: Theory and methods. *J. Log. Program.* 19, 629–679.
- Ng, A.Y., 2004. Feature selection, L1 vs. L2 regularization, and rotational invariance. In: *Proc. of the 21st Int. Conf. on Machine Learning. ICML*, p. 78.
- Nguyen, H.N., Teerakanok, S., Inomata, A., Uehara, T., 2021. The comparison of word embedding techniques in RNNs for vulnerability detection. In: *Proc. of the Int. Conf. on Information Systems Security and Privacy. ICISSP*, pp. 109–120.
- Okun, V., Delaitre, A., Black, P.E., et al., 2013. Report on the static analysis tool exposition (SATE) IV. In: *NIST Special Publication 500*, p. 297.
- Pennington, J., Socher, R., Manning, C.D., 2014. GloVe: Global vectors for word representation. In: *Proc. of the 2014 Conf. on Empirical Methods in Natural Language Processing. EMNLP*, pp. 1532–1543.
- Qiang, G., 2022. Research on software vulnerability detection method based on improved CNN model. *Sci. Program.* 2022 (1), 4442374.
- Qu, M., Tang, J., 2019. Probabilistic logic neural networks for reasoning. *Adv. Neural Inf. Process. Syst.* 32.
- Riegel, R., Gray, A., Luus, F., Khan, N., Makondo, N., Akhalwaya, I.Y., Qian, H., Fagin, R., Barahona, F., Sharma, U., et al., 2020. Logical neural networks. *arXiv preprint arXiv:2006.13155*.
- Russell, R., Kim, L., Hamilton, L., Lazovich, T., Harer, J., Ozdemir, O., Ellingwood, P., McConley, M., 2018. Automated vulnerability detection in source code using deep representation learning. In: *Proc. of the 17th IEEE Int. Conf. on Machine Learning and Applications. ICMLA*, pp. 757–762.
- Sasaki, Y., et al., 2007. The truth of the F-measure. *Teach Tutor Mater* 1 (5), 1–5.
- Shang, Y., Liu, S., 2024. LRNN: A formal logic rules-based neural network for software defect prediction. In: *Proc. of the International Conference on Formal Engineering Methods. ICFEM, Springer*, pp. 106–124.
- Shin, Y., Meneely, A., Williams, L., Osborne, J.A., 2010. Evaluating complexity, code churn, and developer activity metrics as indicators of software vulnerabilities. *IEEE Trans. Softw. Eng.* 37 (6), 772–787.
- Shin, Y., Williams, L., 2008. An empirical model to predict security vulnerabilities using code complexity metrics. In: *Proc. of the 2nd ACM-IEEE Int. Symp. on Empirical Software Engineering and Measurement*. pp. 315–317.
- Streiner, D.L., Norman, G.R., 2006. Precision and accuracy: two terms that are neither. *J. Clin. Epidemiol.* 59 (4), 327–330.
- Wasikowski, M., Chen, X.-w., 2009. Combating the small sample class imbalance problem using feature selection. *IEEE Trans. Knowl. Data Eng.* 22 (10), 1388–1400.
- Wu, Y., Zou, D., Dou, S., Yang, W., Xu, D., Jin, H., 2022. VulCNN: An image-inspired scalable vulnerability detection system. In: *Proc. of the 44th Int. Conf. on Software Engineering. ICSE*, pp. 2365–2376.
- Xu, A., Dai, T., Chen, H., Ming, Z., Li, W., 2018. Vulnerability detection for source code using contextual LSTM. In: *Proc. of the 5th Int. Conf. on Systems and Informatics. ICSAI, IEEE*, pp. 1225–1230.
- Yin, L., Ge, Y., Xiao, K., Wang, X., Quan, X., 2013. Feature selection for high-dimensional imbalanced data. *Neurocomputing* 105, 3–11.

- Yuan, B., Lu, Y., Fang, Y., Wu, Y., Zou, D., Li, Z., Li, Z., Jin, H., 2023. Enhancing deep learning-based vulnerability detection by building behavior graph model. In: Proc. of the 45th IEEE/ACM Int. Conf. on Software Engineering. ICSE, IEEE, pp. 2262–2274.
- Zagane, M., Abdi, M.K., Alenezi, M., 2020. Deep learning for software vulnerabilities detection using code metrics. *IEEE Access* 8, 74562–74570.
- Zhou, Z.-H., 2021. *Machine Learning*. Springer Nature.
- Zhou, P., Hu, X., Li, P., Wu, X., 2017. Online feature selection for high-dimensional class-imbalanced data. *Knowl.-Based Syst.* 136, 187–199.
- Zou, D., Wang, S., Xu, S., Li, Z., Jin, H., 2019. μ VulDeePecker: A deep learning-based system for multiclass vulnerability detection. *IEEE Trans. Dependable Secur. Comput.* 18 (5), 2224–2236.